

Assignment 5 Report

From Supervised Learning to Self-Supervised Learning using SimCLR

Deep Learning – Spring 2026

Student Name: Muhammad Usman Muneer

Roll Number: BSCS22008

GitHub Repository: https://github.com/Sheild007/DL_Assignment5_SimCLR_BSCS22008

Seed: 2026

Date: 31 May 2026

Headline Result	Test accuracy
Supervised ResNet-18 (10 % labels)	68.65 %
Random encoder + linear probe	26.70 %
SimCLR encoder + linear probe	74.68 %
SimCLR encoder + fine-tuning	82.81 %

1 Conceptual Warm-up (Task 0)

Q. In supervised classification, what information does the model learn from?

A. The model learns from *(image, label)* pairs. A loss function (cross-entropy for CIFAR-10) measures how far the network’s output is from the correct label, and back-propagation pushes the weights to reduce that loss. So all the knowledge the model gains comes from the labels acting through the loss function.

Q. If CIFAR-10 labels are removed, do the images still contain useful visual structure? Explain.

A. Yes. Even without labels, the pixels themselves carry strong patterns: shapes, textures, colours, edges, and object parts. A bird and a frog look different in pixel space regardless of whether anyone has named them. Self-supervised learning exploits this structure.

Q. If two augmented versions are created from the same original image, should their feature representations be similar or different?

A. They should be *similar*. Cropping, flipping, or jittering colours does not change the underlying object, so a good feature extractor should be invariant to these nuisances and produce nearby vectors for the two views.

Q. If two images come from different original images, should their feature representations be similar or different?

A. They should be *different* (or at least more different than two augmentations of the same image). Different images depict different content, so their features should sit in different parts of the embedding space.

Q. Why might a model trained without labels still be useful for classification later?

A. Because the encoder learns *which images look alike*, it groups visually similar images near each other in feature space. Classes are visually similar by definition, so a tiny labelled dataset is enough to learn a linear boundary between those already-formed groups. The encoder does the hard work; the classifier just picks names.

Q. What is the difference between pretraining and fine-tuning?

A. *Pretraining* is the first phase: train the encoder on a large unlabelled dataset using a generic objective such as NT-Xent. *Fine-tuning* is the second phase: take the pretrained weights as initialisation and continue training, usually with labels and a smaller learning rate, on the downstream task. Pretraining gives a good starting point; fine-tuning adapts it to the specific task.

2 Supervised Baseline (Task 1)

Setup. ResNet-18 (CIFAR-modified: 3×3 conv1, no max-pool, 512-d output) trained from scratch with cross-entropy on the fixed 10 % labelled split for 20 epochs. Best validation checkpoint was kept.

Results.

- **Best validation accuracy:** 69.02 % at epoch 16.
- **Test accuracy:** 68.65 %.
- Training loss falls smoothly from 1.82 $\rightarrow$ 0.57; validation loss bottoms at 0.93 (epoch 16) and then drifts up, signalling mild overfitting beyond epoch 16, which is expected with only 5 000 labelled images.

Item	Value Used
Batch size	64
SimCLR epochs	50
Linear probing epochs	20
Fine-tuning epochs	20
Learning rate	3×10^{-4}
Optimizer	Adam
Temperature (τ)	0.5
Projection dimension	128
Encoder	ResNet-18 modified for CIFAR-10
GPU/CPU used	2× NVIDIA T4 (Kaggle), DataParallel
Approx. training time	SimCLR 70 min, Task 1 / 6 / 7 each $\approx$ 5 min
Random seed	2026

Table 1: Training hyper-parameters. All values match Section 6 of the assignment.

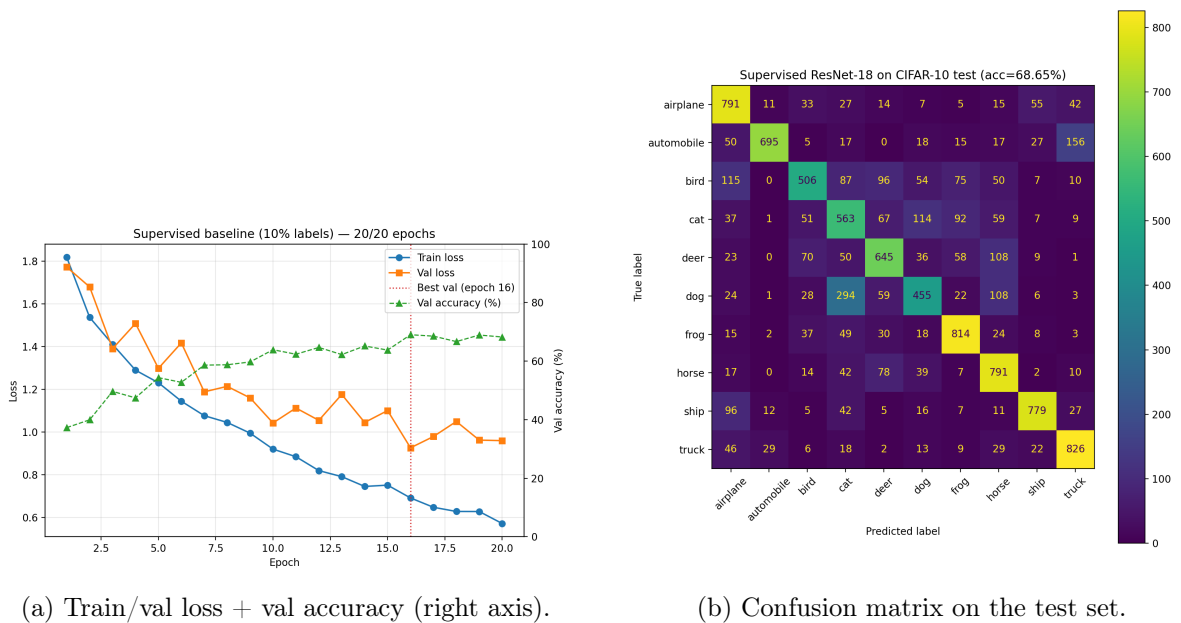


Figure 1: Supervised baseline trained from scratch on 10 % labels.

What the confusion matrix tells us. The model handles *vehicles* (automobile, ship, truck) better than *animals*. The two confusing pairs are *cat* $\leftrightarrow$ *dog* and *bird* $\leftrightarrow$ *deer/frog*, which share visually similar fur or feather textures and are hard to separate with limited training data.

Model	Test Accuracy
Supervised ResNet-18 from scratch using 10 % labels	68.65 %

3 Augmentation Visualization (Task 2)

The SimCLR augmentation pipeline (random resized crop, horizontal flip, colour jitter, random grayscale, normalize) is applied *twice* per image to build a positive pair. The visualization below shows 10 examples (*Original* | *View 1* | *View 2*).



Q&A on augmentations. Q. Are the two augmented views identical?

A. No. Each view has a different random crop, flip, and colour jitter. The randomness is what makes them different.

Q. Do they still represent the same object?

A. Yes. Despite different colours and crops, the object identity (a bird is still a bird, a truck is still a truck) is preserved.

Q. Why should SimCLR treat them as a positive pair?

A. Because they share the same underlying content. Forcing their features to be close teaches the encoder to ignore irrelevant nuisances (colour, crop, flip) and focus on object-level features.

Q. What could go wrong if augmentations are too weak?

A. The two views become almost identical. The model can satisfy NT-Xent by learning a near-identity mapping that copies low-level pixel statistics. The features won't generalise.

Q. What could go wrong if augmentations are too strong?

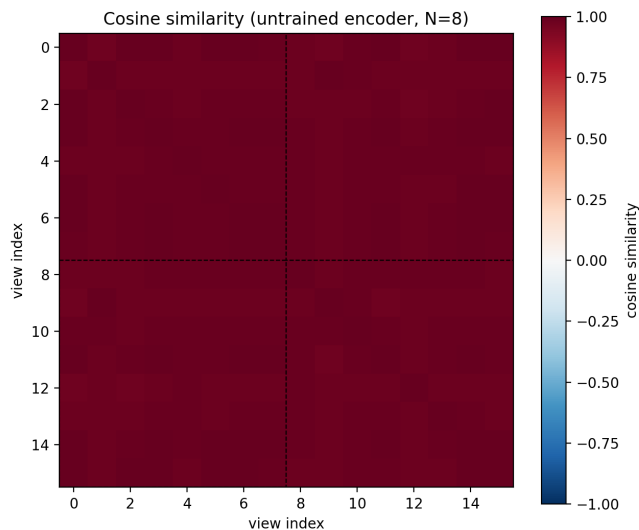
A. The views may no longer share content (e.g. a tiny grey crop containing only background). The positive pair becomes wrong, and pulling those views together would actively hurt the representation.

4 Feature Similarity Before Training (Task 3)

Cosine similarity between features of an *untrained*, randomly-initialised ResNet-18 on a batch of 64 validation images:

Pair Type	Avg. Cosine Similarity (Before)
Same image, two augmented views	0.987
Different images	0.984
Gap (same – different)	+0.003

Reading. Both numbers are extremely high (≈ 0.99) and almost equal. A random ReLU encoder with He initialisation maps every image to roughly the same direction in feature space, so the network at this point cannot tell “same image” from “different images.” The whole point of SimCLR is to open up that gap.



5 SimCLR Implementation (Task 4)

5.1 Encoder + Projection Head

The encoder is a CIFAR-modified ResNet-18 producing a 512-d feature h . The projection head is $\text{Linear}(512 \rightarrow 256) \rightarrow \text{ReLU} \rightarrow \text{Linear}(256 \rightarrow 128)$ producing $z = g(h)$. Crucially, z is used *only during pretraining*; downstream tasks (probe, fine-tune, t-SNE) use the 512-d h .

5.2 Positive and Negative Pairs

For a batch of $N = 4$ original images, two augmentations yield $2N = 8$ views.

Original Image	View 1 Index	View 2 Index	Positive Pair?
image 0	0	4	yes
image 1	1	5	yes
image 2	2	6	yes
image 3	3	7	yes

Table 2: Concatenation order: $[\text{view-1}_0, \dots, \text{view-1}_{N-1}, \text{view-2}_0, \dots, \text{view-2}_{N-1}]$. The positive partner of index $i < N$ is $i + N$, and for $i \geq N$ it is $i - N$.

5.3 Similarity Matrix and NT-Xent

After L2-normalising the projections, I compute $\mathbf{S} = \mathbf{Z}\mathbf{Z}^\top \in \mathbb{R}^{2N \times 2N}$. Diagonal entries (a view compared with itself) are masked to $-\infty$. NT-Xent then becomes a multi-class cross-entropy where the target for row i is the index of its positive partner:

$$\mathcal{L}_i = -\log \frac{\exp(s_{i,\text{pos}(i)}/\tau)}{\sum_{k \neq i} \exp(s_{i,k}/\tau)}, \quad \tau = 0.5.$$

Q&A on Task 4.3. Q. Why is the diagonal ignored?

A. A view’s similarity with itself is always 1.0 by construction, contributing nothing useful and dominating the soft-max. Masking it removes a trivial true-positive that would otherwise leak into the loss.

Q. Where are the positive pairs located?

A. On the two off-diagonals at offset $\pm N$: positions $(i, i + N)$ and $(i + N, i)$ for $0 \leq i < N$. With $N = 8$ this is the secondary diagonal of the top-right and bottom-left quadrants.

Q. Why are all other entries treated as negatives?

A. Without labels we cannot tell semantically related images apart, so SimCLR makes the simple assumption that any other image in the batch is a different concept. This produces $2N - 2$ negatives per anchor and is what gives the loss its strong contrastive signal.

6 SimCLR Pretraining Results (Task 5)

50 epochs on the unlabeled split, batch 64 ($2N=128$ views), Adam lr 3×10^{-4} , $\tau=0.5$. NT-Xent loss falls smoothly from 3.85 to 3.21 with no instabilities (single GPU, no DDP / no all-gather).

Before vs. after similarity.

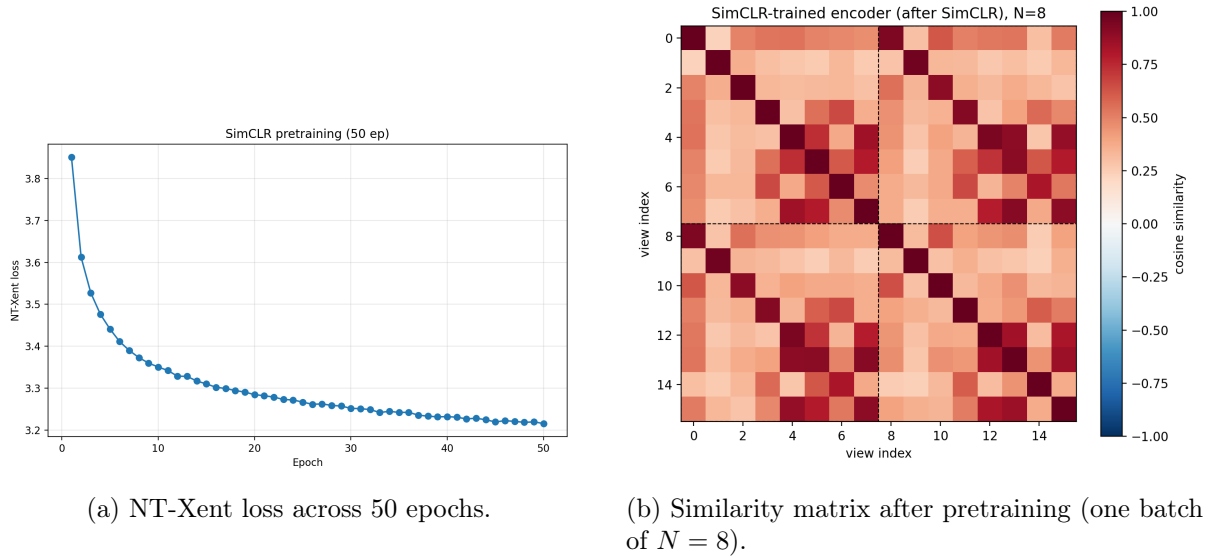


Figure 2: SimCLR pretraining outcomes.

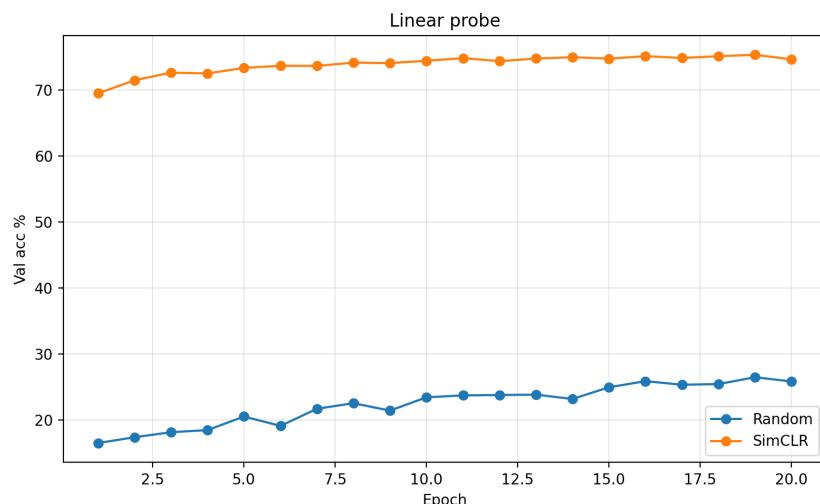
Pair Type	Before SimCLR	After SimCLR
Same image, two augmented views	0.987	0.908
Different images	0.984	0.434
Gap (same – different)	+0.003	+0.474

Table 3: Mean cosine similarity on a fixed val batch.

Discussion. Two views of the *same* image stay highly similar (0.91), while *different* images now sit much further apart (0.43). The gap grew from ≈ 0 to ≈ 0.47 , which is exactly what NT-Xent is designed to do. The same-pair similarity dropped slightly from 0.99 to 0.91 because the encoder is no longer collapsing every input to one direction; it has spread the data across the unit hyper-sphere while still keeping augmented twins close. This is the textbook SimCLR result.

7 Linear Probe Evaluation (Task 6)

Both probes train only Linear($512 \rightarrow 10$) for 20 epochs (Adam lr 3×10^{-4}); the encoder is frozen.

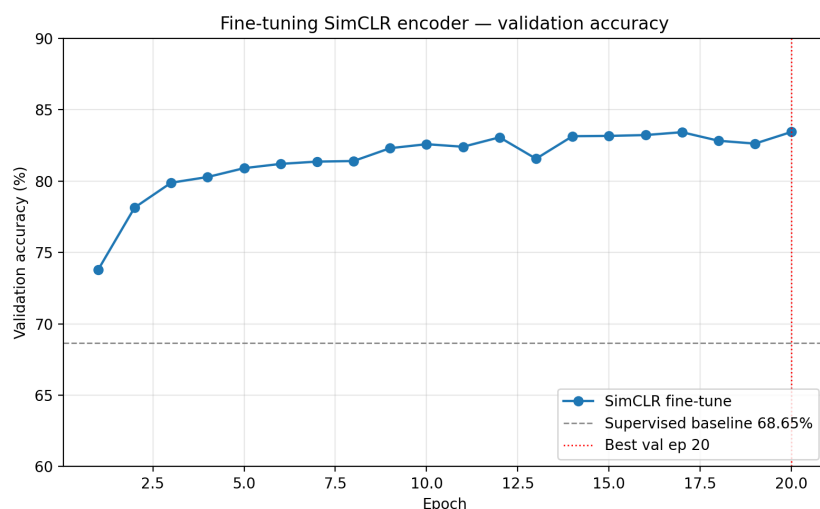


Model	Encoder	Trainable Part	Test Accuracy
Random Linear Probe	Random frozen encoder	Linear classifier only	26.70 %
SimCLR Linear Probe	SimCLR frozen encoder	Linear classifier only	74.68 %

Reading. A linear classifier cannot fix bad features. A 48-percentage-point gap (26.7% → 74.7%) therefore means the SimCLR features are *linearly separable into CIFAR-10 classes*, even though SimCLR never saw a single label. The random probe also climbs slowly, which is expected because random ResNet features still preserve some low-frequency colour and texture statistics that loosely correlate with class.

8 Fine-tuning Results (Task 7)

Encoder initialised from `simclr_encoder.pt`, then *all* parameters trained for 20 epochs with cross-entropy on the 10 % labelled split.



The fine-tuned model surpasses the supervised baseline within a single epoch (73.78% vs 37.30%), and reaches its peak at epoch 20 (val 83.44%, test 82.81%).

Experiment	Test Accuracy
Supervised ResNet-18 from scratch using 10 % labels	68.65 %
Random frozen encoder + linear classifier	26.70 %
SimCLR frozen encoder + linear classifier	74.68 %
SimCLR pretrained encoder + full fine-tuning	82.81 %

Table 4: Final comparison of all four pipelines.

Why fine-tuning beats the linear probe by ~ 8 pp. The probe is restricted to a hyper-plane in the SimCLR feature space; fine-tuning lets every layer of the encoder adapt to the new (now *labelled*) objective, smoothing out class boundaries that contrastive learning had to leave fuzzy.

Why fine-tuning beats supervised-from-scratch by ~ 14 pp. With only 5 000 labelled images, training all 11M parameters from random init mostly underfits and overfits in turns. Starting from features that already organise the manifold by visual content gives the optimiser a much better basin to operate in.

9 PCA / t-SNE Visualization (Task 8)

t-SNE projections of 512-d encoder features for 1 000 fixed validation images (seed 2026, coloured by class label; labels used *only* for colouring). t-SNE is preferred over PCA here because it preserves *local* neighbourhoods, so semantically similar images that lie on a curved manifold appear as tight clusters instead of being squashed by linear projection.

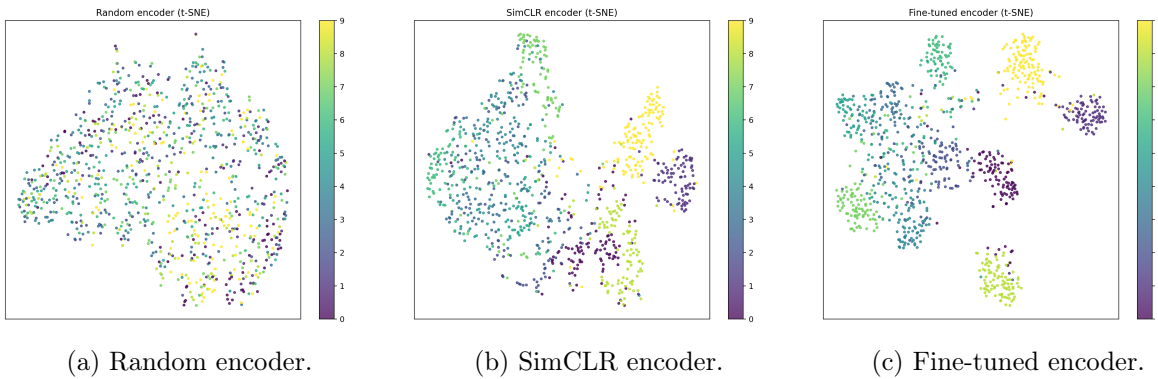


Figure 3: t-SNE visualisations of the 512-d feature space.

Q. Do features from the random encoder show class-wise grouping?

A. No. All ten colours are mixed inside one blob, so the random encoder has no notion of class.

Q. Do features from the SimCLR encoder show better grouping?

A. Yes. Distinct lobes form for trucks (yellow, top-right) and ships (dark purple, right), and animal classes start splitting along the lower edge, all without ever seeing a label.

Q. Does fine-tuning improve class separation?

A. Yes. Vehicle classes (truck, ship, automobile) form tight, well-separated clusters, deer and frog detach into their own lobes, and the central animal mass starts dividing into recognisable sub-groups. The 75 % linear-probe and 83 % fine-tune accuracies are the quantitative confirmation.

Q. Which classes are still confused?

A. The animal trio {cat, dog, bird} remains partially overlapped in the central region, matching the off-diagonal mass in the supervised confusion matrix. These are genuinely visually similar at 32×32 resolution.

10 Final Comparison Table

Model	Uses Labels During Pretraining?	Encoder Frozen?	Test Acc
Supervised ResNet-18 (10 % labels)	Yes	No	68.65
Random encoder + linear probe	No	Yes	26.70
SimCLR encoder + linear probe	No	Yes	74.68
SimCLR encoder + fine-tuning	No (only at fine-tune)	No	82.81

11 Discussion: What SimCLR Learned

The before/after similarity table and the three PCA plots tell the same story from two angles.

- Before training, the encoder maps everything to one tight cap of the unit sphere. Cosine similarities are ≈ 0.99 everywhere, so it cannot distinguish anything.
- After 50 epochs of NT-Xent, the encoder has *spread* the points across the sphere while keeping augmentations of one image close together. Same-pair similarity stays high (0.91); different-image similarity drops to 0.43, a 0.47 gap.
- This geometric organisation is what makes the linear probe jump from 26 % (random) to 75 % (SimCLR). SimCLR did not learn “cat-ness” or “truck-ness”; it learned a representation in which visually similar things are nearby. CIFAR-10 classes happen to be visually coherent, so a linear classifier can carve them out.
- Fine-tuning adds the missing 8 percentage points: it lets every layer adjust its decision boundary to the labels, but starting from an already well-organised feature space. This is why fine-tuning needs far fewer labels than from-scratch training.

12 Implementation Issues and Limitations

- **Single-GPU SimCLR.** Kaggle provides $2 \times$ T4 but `nn.DataParallel` naively splits the loss per GPU and breaks NT-Xent (each GPU sees half the negatives). I therefore pretrained on `cuda:0` only with batch 64; Tasks 1 and 7 still use both GPUs because cross-entropy is per-image.
- **Batch size 64.** The assignment recommended 64; I kept it. Larger batches (128/256) usually improve SimCLR by giving more negatives, but T4 memory and the single-GPU constraint above made 64 a safer choice.
- **Mild overfitting in Task 1.** Training loss keeps falling after epoch 16 while val loss rises, which is typical for 10 % labels. The best-checkpoint mechanism keeps the epoch-16 weights so the reported number is the genuine best.
- **Class confusions.** The animal classes (especially cat/dog and bird/deer) remain harder than vehicles. This is consistent with the raw pixel similarity at 32×32 resolution and is not a SimCLR failure.

- **2D visualisation loses information.** PCA is linear; t-SNE would tighten the clusters visually. The clean accuracy numbers (83% fine-tune) are the better measure of what the encoder learned.

Appendix: metrics.json

```
{
  "student_name": "Muhammad Usman Muneer",
  "roll_number": "BSCS22008",
  "seed": 2026,
  "batch_size": 64,
  "simclr_epochs": 50,
  "linear_probe_epochs": 20,
  "finetuning_epochs": 20,
  "learning_rate": 0.0003,
  "temperature": 0.5,

  "supervised_10percent_test_acc": 0.6865,
  "random_linear_probe_test_acc": 0.2670,
  "simclr_linear_probe_test_acc": 0.7468,
  "simclr_finetune_test_acc": 0.8281,

  "same_view_similarity_before": 0.9870,
  "different_image_similarity_before": 0.9836,
  "same_view_similarity_after": 0.9079,
  "different_image_similarity_after": 0.4336,

  "github_repo_url": "https://github.com/Sheild007/
    DL_Assignment5_SimCLR_BSCS22008",
  "first_commit_date": "2026-05-20",
  "last_commit_before_deadline": "2026-05-31",
  "number_of_meaningful_commits": 27
}
```